# Edit Text Files from the Shell Prompt

## Objectives

Create and edit text files from the command line with the `vim` editor.

## Edit Files with Vim

The fundamental design principle of Linux is that it supports storage of the information and configuration settings in text-based files. These files follow various structures such as lists of settings, INI-like formats, structured XML or YAML, and others. The advantage of storing files in a text-based structure is that they are easily edited with any simple text editor.

Vim is an improved version of the `vi` editor, which is distributed with Linux and UNIX systems. Vim is highly configurable and efficient for practiced users, including split-screen editing, color formatting, and highlighting for editing text.

## Benefits of the Vim Editor

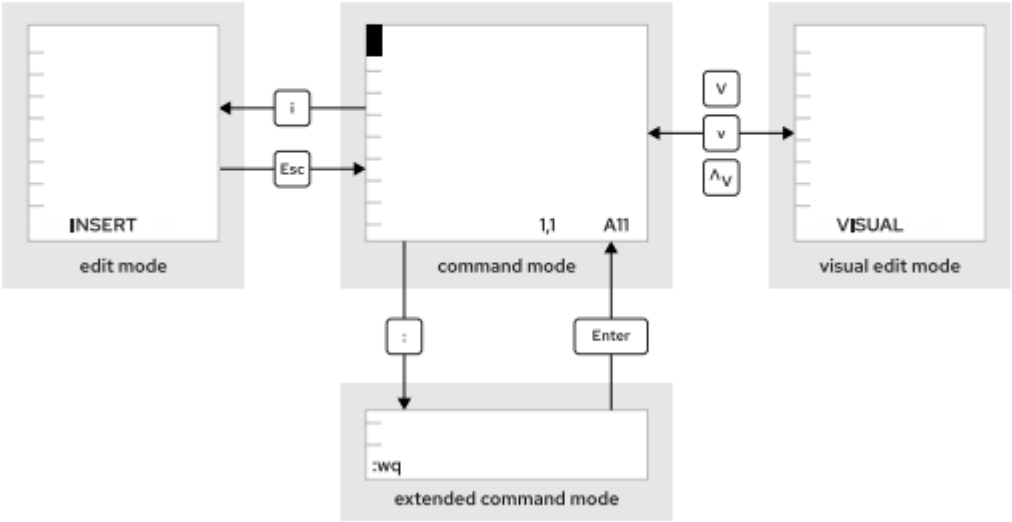
When a system uses a text-only shell prompt, you should know how to use at least one text editor for editing files. You can then edit text-based configuration files from a terminal window or remote logins through the `ssh` command or the Web Console. You also do not need access to a graphical desktop to edit files on a server, and that server might not need to run a graphical desktop environment.

The key reason to learn Vim is that it is almost always installed by default on a server for editing text-based files. The *Portable Operating System Interface* or *POSIX* standard specified the `vi` editor on Linux, and many other UNIX-like operating systems largely do likewise.

Vim is also used often as the `vi` implementation on other standard operating systems or distributions. For example, macOS currently includes a lightweight installation of Vim by default. So, Vim skills that are learned for Linux might also prove useful elsewhere.

## Vim Operating Modes

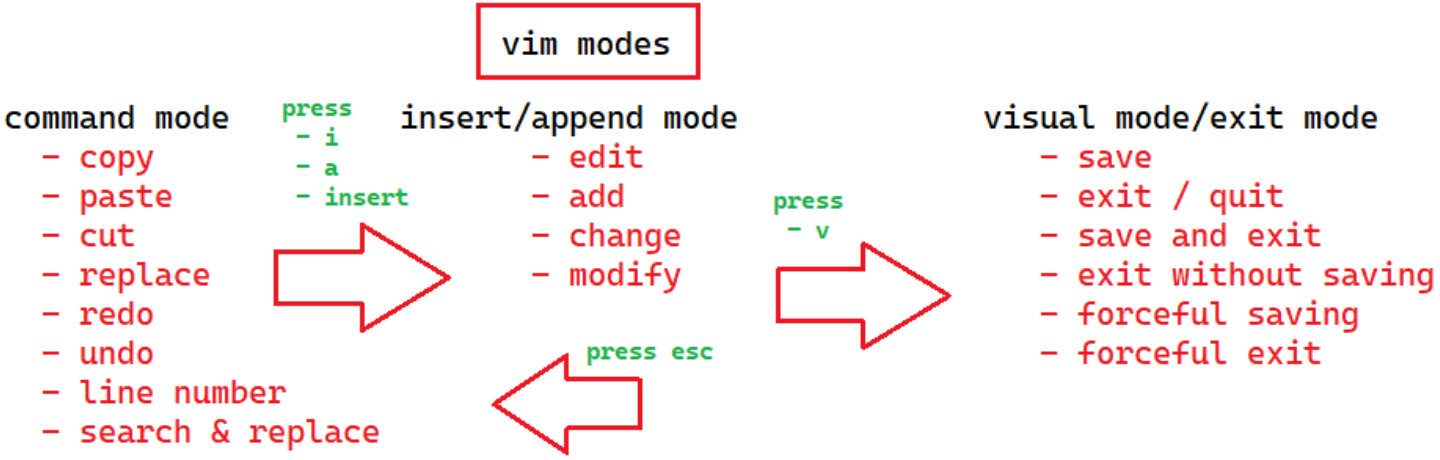
The Vim editor offers various modes of operation such as *command mode*, *extended command mode*, *edit mode*, and *visual mode*. You should always check the current mode as a Vim user, because keystrokes have different effects in different modes.



When you first open Vim, it starts in *command mode*, which is used for navigation, cut and paste, and other text modification. Pressing the required keystroke accesses specific editing functions.

- An `i` keystroke enters *insert mode*, where all typed text becomes file content. Pressing `Esc` returns to command mode.
- A `v` keystroke enters *visual mode*, where multiple characters might be selected for text manipulation. Use `Shift+V` for multiline and `Ctrl+V` for block selection. To exit the visual mode, use the `v`, `Shift+V`, or `Ctrl+V` keystrokes.
- The `:` keystroke begins *extended command mode* for tasks such as writing the file (to save it) and quitting the Vim editor.

| vi | vim |
|---|---|
| 1. b & w | 1. colorful(but colors are for system files) |
| 2. no error checking | 2. has error checking |
| 3. older version | 3. newer version |
| 4. available in both minimal & GUI installation | 4. Not available by default in minimal installation |




Labs :-

1. Creating a file ⇒ $ vim debayan
2. To save and exit ⇒ esc → :wq!

3. To bring line number ⇒ :se nu
4. To remove line number ⇒ :se nonu
5. Line wise cursor movement ⇒ :line_number
6. To go to top line ⇒ press gg
7. To go to the last line → press G
8. Word searching ⇒ /word
9. To replace a word ⇒ :%s/old_word/new_word/g
10. Copy a line ⇒ go to the start of the line & press yy
11. To paste ⇒ press p where u want to paste
12. Multiple line copy ⇒ press no of lines & yy
13. Multiple word copy ⇒ press no of words yw
14. To go to last line ⇒ press G
15. To paste multiple times ⇒ no of time p
16. Line delete ⇒ press dd
17. Multiple line delete ⇒ no of lines dd
18. Undo ⇒ press u
19. Redo ⇒ ctrl+r